Adaptive coordination for multi-component systems

Requirement Change Management

SEAtS | 639986

University of Southampton

Intended Learning Outcomes

By the end you will be able to:

1. Map why requirements shift and trace the resulting ripples across system architecture, stakeholder contracts, and development workflows.
2. Compare predictive, iterative, Agile, and hybrid SDLC models, explaining when each manages change most effectively.
3. Design governance strategies (traceability, systemic mapping, CCBs) that keep change visible and testable.
4. Integrate data management, Git workflows, and automation to build socio-technical feedback loops that stay audit-ready.
5. Synthesize these practices into pragmatic guidance for multi-component programmes operating under evolving constraints.

© University of Southampton

Agenda

1. Requirement change context + systemic drivers.
2. Predictive foundations: Waterfall, V-Model, RUP.
3. Risk-aware & iterative models: Spiral, Incremental, RAD, Iterative, Prototype.
4. Agile frameworks (Scrum, XP, Kanban) + hybrid coordination.
5. Governance levers: traceability, selection compass, stakeholder integration.
6. Tooling feedback loops: data management, Git, reviews, bots.

© University of Southampton

Introduction: Why Requirement Change Matters

Projects rarely proceed exactly as planned. Requirements evolve due to shifting stakeholder needs, technology advances, regulatory changes, or market pressures. Multi-component systems amplify this volatility because an adjustment in one layer ripples through others.

Large-scale system optimisation depends not only on technical robustness but also on managing evolving constraints. Requirement change management therefore becomes less about rigid control and more about adaptive coordination that protects architectural stability.

© University of Southampton

Planning in the Face of Uncertainty

Traditional planning tools—Gantt charts, Work Breakdown Structures, dependency maps—define baselines but assume relative stability. Practical experience shows that rigid tools fail in complex environments; flexibility and reflexivity are crucial.

Hybrid strategy

- Predictive planning: fixed milestones, budgets, compliance checkpoints.
- Adaptive planning: buffers, contingency pathways, iterative review points.

© University of Southampton

Sources and Types of Requirement Change

Requirement change drivers:

- Stakeholders: new user expectations, policy updates.
- Technology shifts: new standards, component obsolescence.
- Integration issues: hidden dependencies discovered late.
- External environment: regulatory, geopolitical, market pressures.

Change types:

- Additive: new requirements enter scope.
- Modificative: parameters are revised.
- Subtractive: removing requirements to regain viability.

Industry case work shows monolithic architectures stay brittle, while decoupled designs with clear interfaces and meta-models absorb shifts more gracefully.

© University of Southampton

Managing Requirement Change

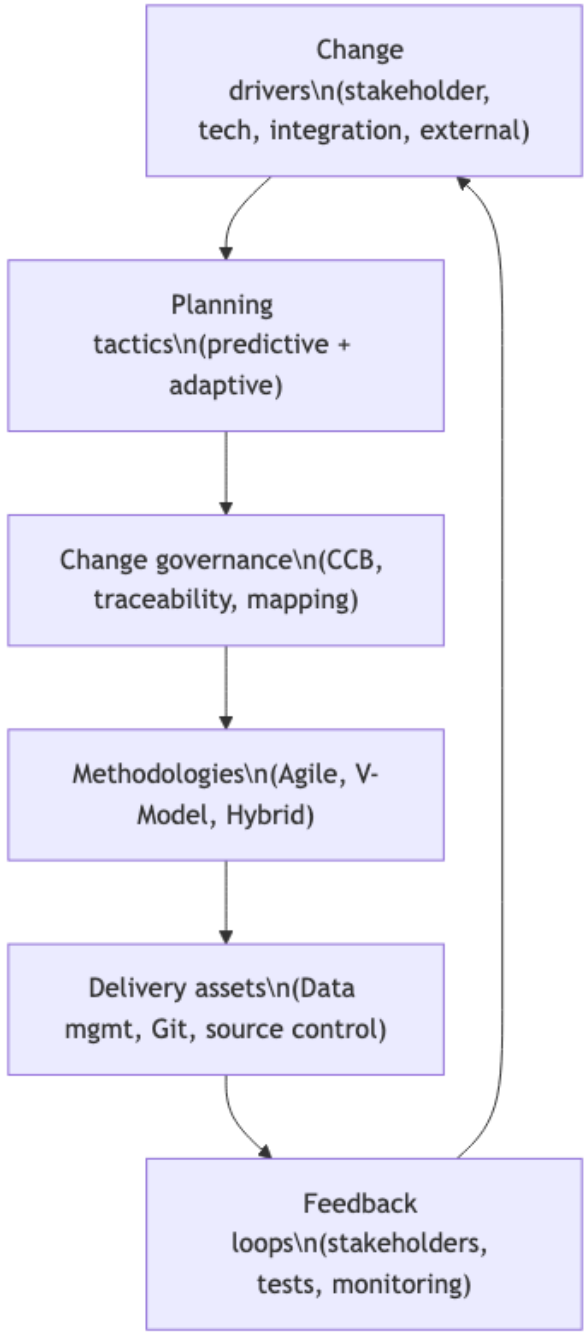
Change Control Boards (CCB)
Formal evaluation before approving any change keeps risk visible and stakeholders aligned.

Systemic mapping
Causal loop diagrams reveal second-order effects and hidden leverage points.

Traceability matrices
Link requirements to design artefacts so impact analysis is fast and auditable.

Optimisation & trade-off analysis
Multi-objective evaluation balances cost, performance, and schedule risk before committing.

© University of Southampton

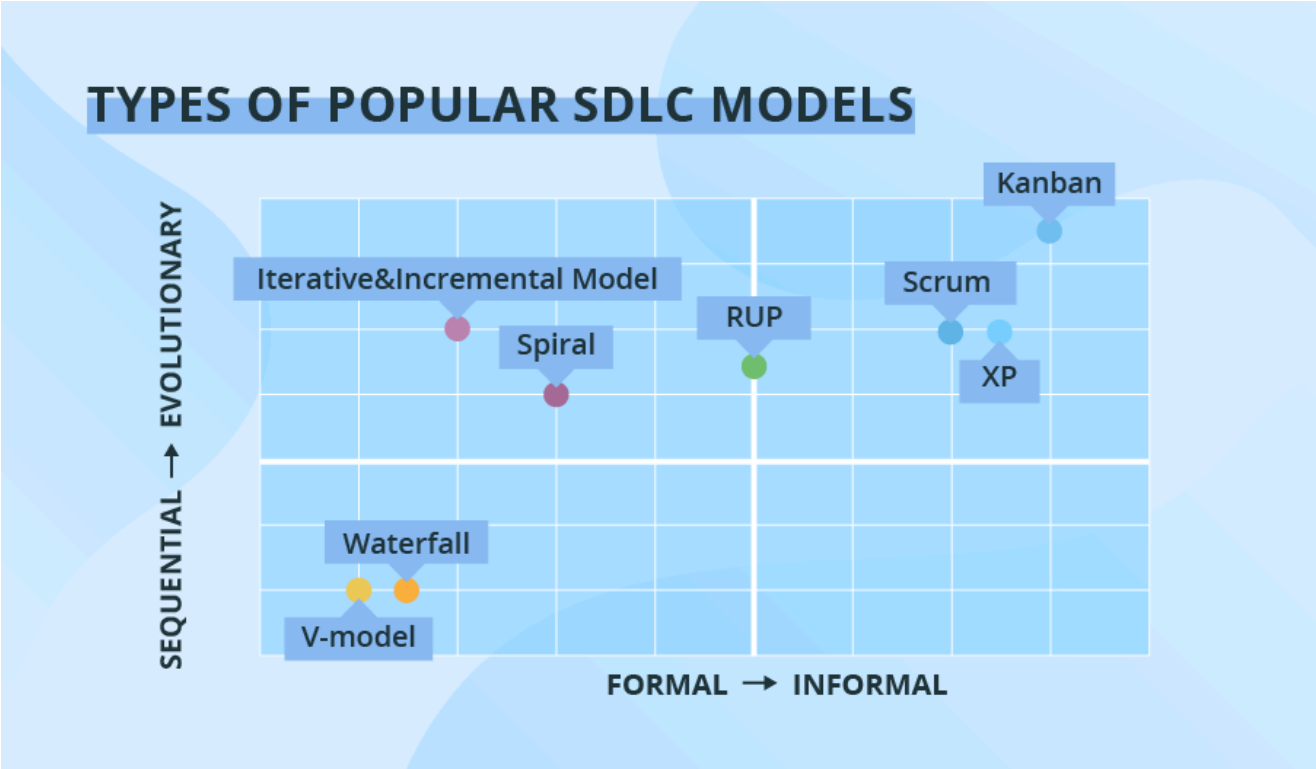


Integrating Systems Thinking and Stakeholder Perspectives

Systemic design highlights ten mindsets (acting locally, embracing variety, envisioning preferred futures, etc.). Applied to change management:

- Stakeholder analysis evolves as actors reinterpret project outcomes—treat it as a living artefact.
- Feedback loops (positive + negative) must be modelled to observe reinforcing dynamics like cost overruns driving scope reduction.
- Causal maps connect requirement shifts to architecture, testing, and socio-technical consequences.

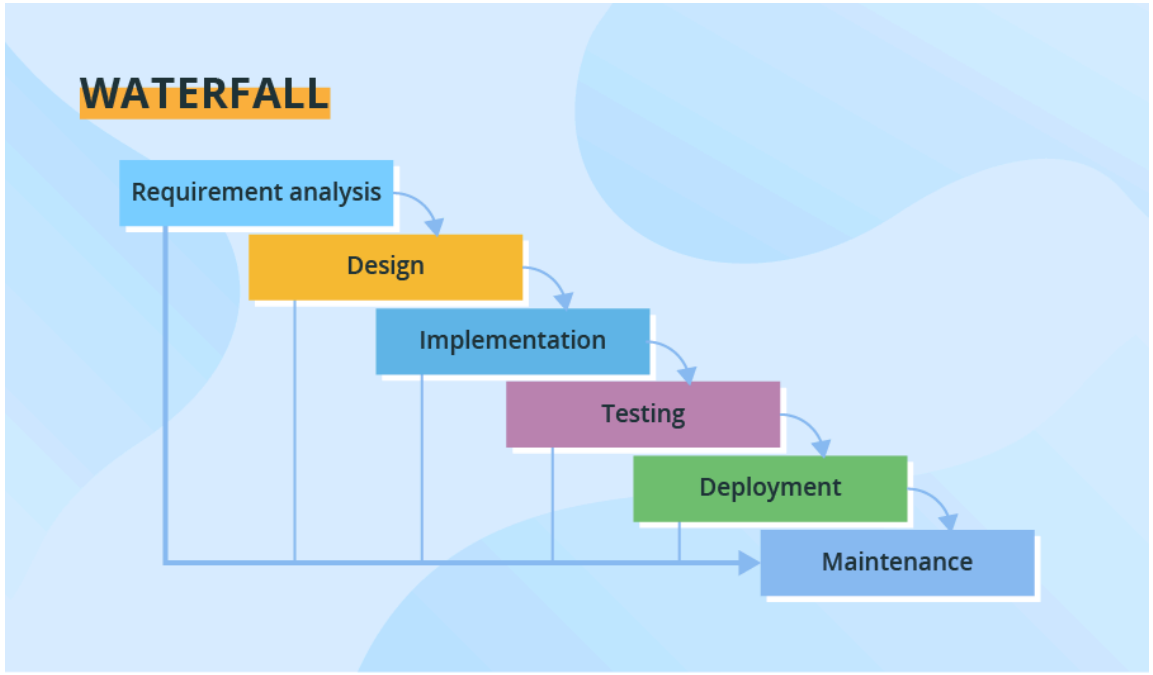
© University of Southampton



Formal vs Adaptive

- **Formal (predictive):** Essential in safety-critical domains (aviation, nuclear) where certification, traceability, and locked baselines dominate.
- **Adaptive (systemic):** Better for innovation-led programmes (software platforms) where iterative co-design with stakeholders reveals emerging needs.
- **Blended approaches:** Agile Systems Engineering combines iteration, stakeholder co-design, and continuous integration while keeping regulated artefacts up to date.

© University of Southampton



Waterfall Model

1. **Requirements & feasibility:** Capture functional/non-functional needs, constraints, and viability evidence up front.
2. **System design:** Produce high-level architecture plus detailed component contracts.
3. **Implementation:** Build modules per spec, integrating them sequentially.
4. **Testing:** Run unit → integration → system → acceptance tests only after build finishes.
5. **Deployment:** Release and configure the system in production.
6. **Maintenance:** Deliver fixes, enhancements, and performance updates post-launch.

© University of Southampton

Waterfall Model – Pros & Cons

Advantages

- Linear flow makes progress easy to track.
- Each phase has tangible deliverables/milestones.
- Stable requirements benefit from upfront clarity.
- Predictable budgets/schedules satisfy procurement expectations.

Disadvantages

- Late change is expensive; feedback arrives only after build/test.
- Limited customer interaction encourages misunderstandings.
- Defects discovered in final phases ripple backwards.

Best for

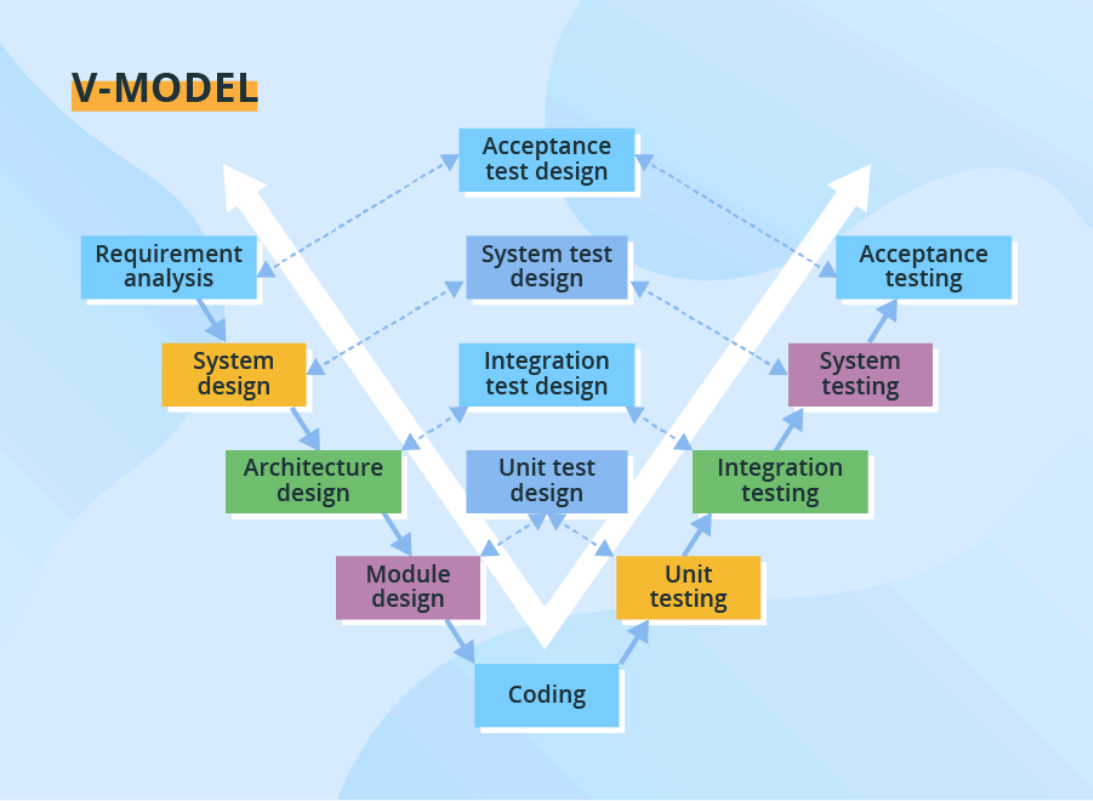
- Short (<12 month) initiatives with mature tech stacks and fixed scope.
- Regulated bids that demand exhaustive documentation and sequential sign-offs.

Avoid when

- Stakeholders need incremental value or requirements shift frequently.

Large enterprise surveys still show ~40% Waterfall use for public-sector, defense, and compliance-heavy programmes despite the Agile surge.

© University of Southampton



V-Model

The V-Model extends Waterfall by emphasising verification and validation at every stage. The left side of the "V" captures requirements, system design, and component design; the right side mirrors those stages with unit tests, integration tests, system tests, and acceptance validation that align directly back to documented requirements.

Phase pairings

1. Requirement analysis ↔ Acceptance testing.
2. System design ↔ System testing.
3. Architectural design ↔ Integration testing.
4. Module design ↔ Unit testing.
5. Coding sits at the base, linking design intent to verification evidence.

© University of Southampton

V-Model – Strengths & Weaknesses

Strengths

- Strong traceability from requirement to validation.
- Clear documentation and milestone checkpoints.
- Embedded QA lowers defect cost and eases certification.

Best for

- Mission/safety-critical systems (automotive, avionics, medical, rail).
- Contracts requiring formal verification & validation evidence.

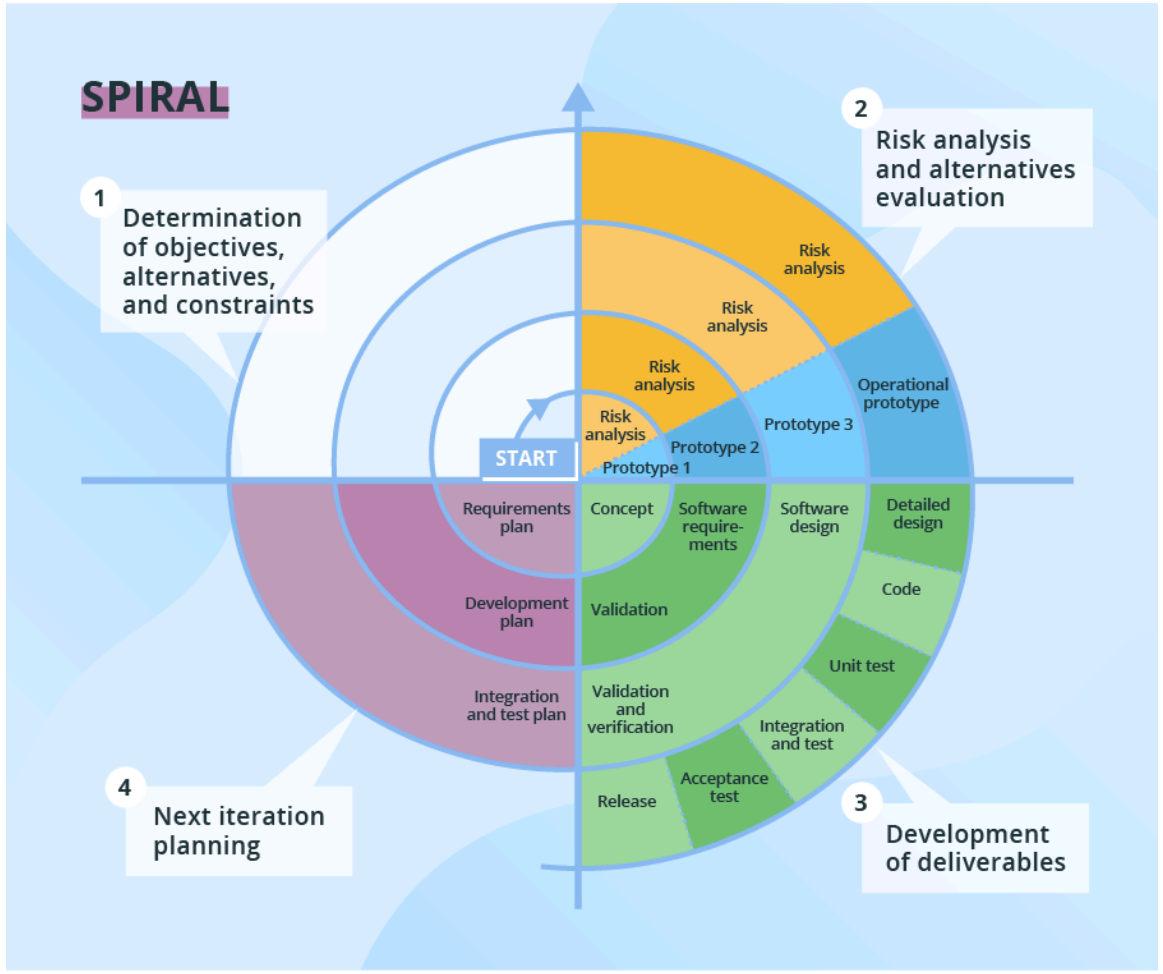
Weaknesses

- Inflexible when requirements shift mid-cycle.
- Late requirement errors surface only after costly phases.
- Heavy upfront planning delays the first build.

Avoid when

- Requirements evolve quickly or small teams need lightweight governance.

© University of Southampton



Spiral Model – Flow

Each loop of the spiral executes four tasks:

1. **Planning:** Define objectives, constraints, and alternatives.
2. **Risk analysis:** Identify/mitigate technical, schedule, or stakeholder risks early.
3. **Engineering:** Design, prototype, and build the next level of functionality.
4. **Evaluation:** Engage users, assess results, and decide the next loop's goals.

© University of Southampton

Spiral Model – Pros & Cons

Advantages

- Explicit risk analysis prevents costly surprises.
- Iterative loops accommodate evolving requirements.
- Customer feedback is built into every cycle.

Disadvantages

- Complex to manage and often costlier than simpler models.
- Requires mature risk assessment skills.
- Scope creep is likely without disciplined loop planning.

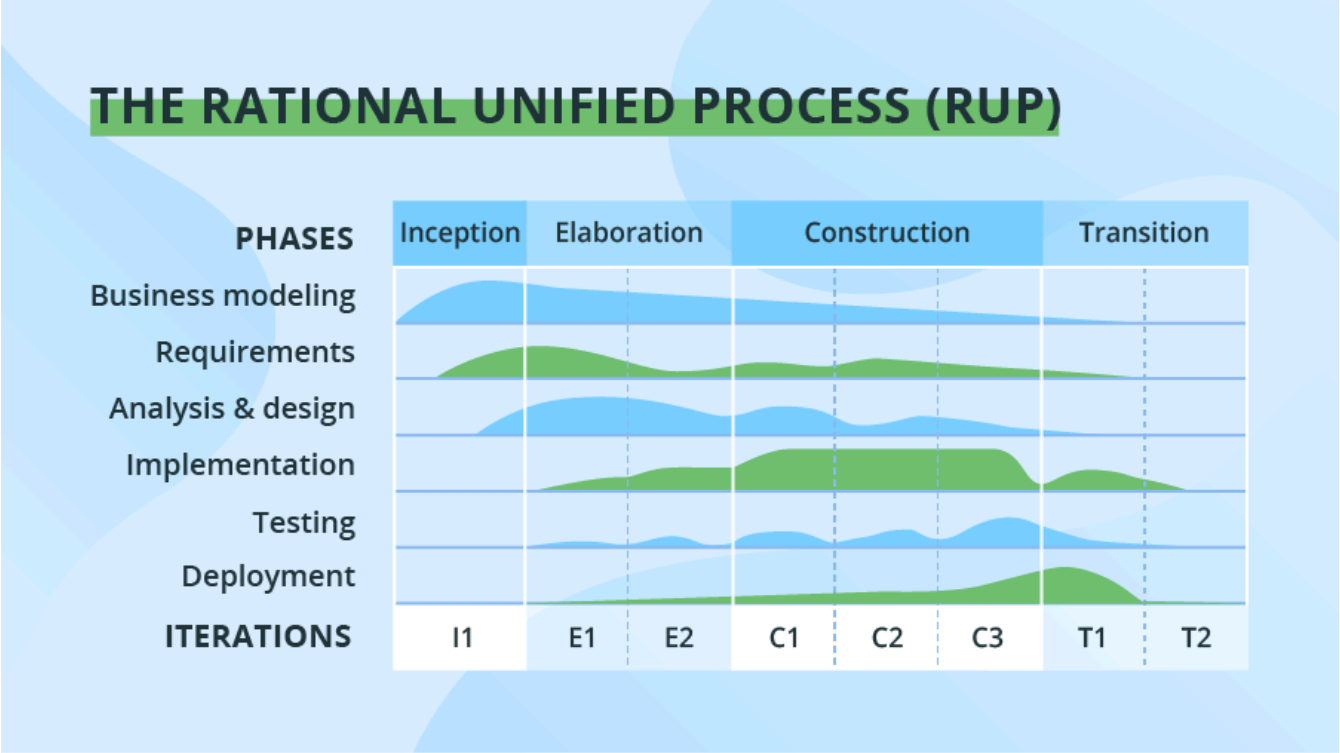
Best for

- Large, high-risk initiatives (ERP refresh, telecom control, online banking cores).
- R&D or experimental programmes with evolving feasibility evidence.

Avoid when

- Requirements are clear, risk is low, or budgets cannot support repeated cycles.

© University of Southampton



Rational Unified Process (RUP)

- RUP blends linear phases (inception → elaboration → construction → transition) with iterative cycles inside each phase.
- Core disciplines—requirements, design, implementation, verification, deployment—run in parallel but with varying intensity, enabling traceability and flexibility simultaneously.

© University of Southampton

Rational Unified Process (RUP) – Pros & Cons

Benefits

- Component-based, architecture-centric approach minimises rework.
- Phase-specific documentation keeps decision logs audit-ready.
- Scales across large, multi-team programmes with shared governance.

Challenges

- Extensive documentation and overlapping tasks extend schedules.
- Requires strong project management and architecture leadership.

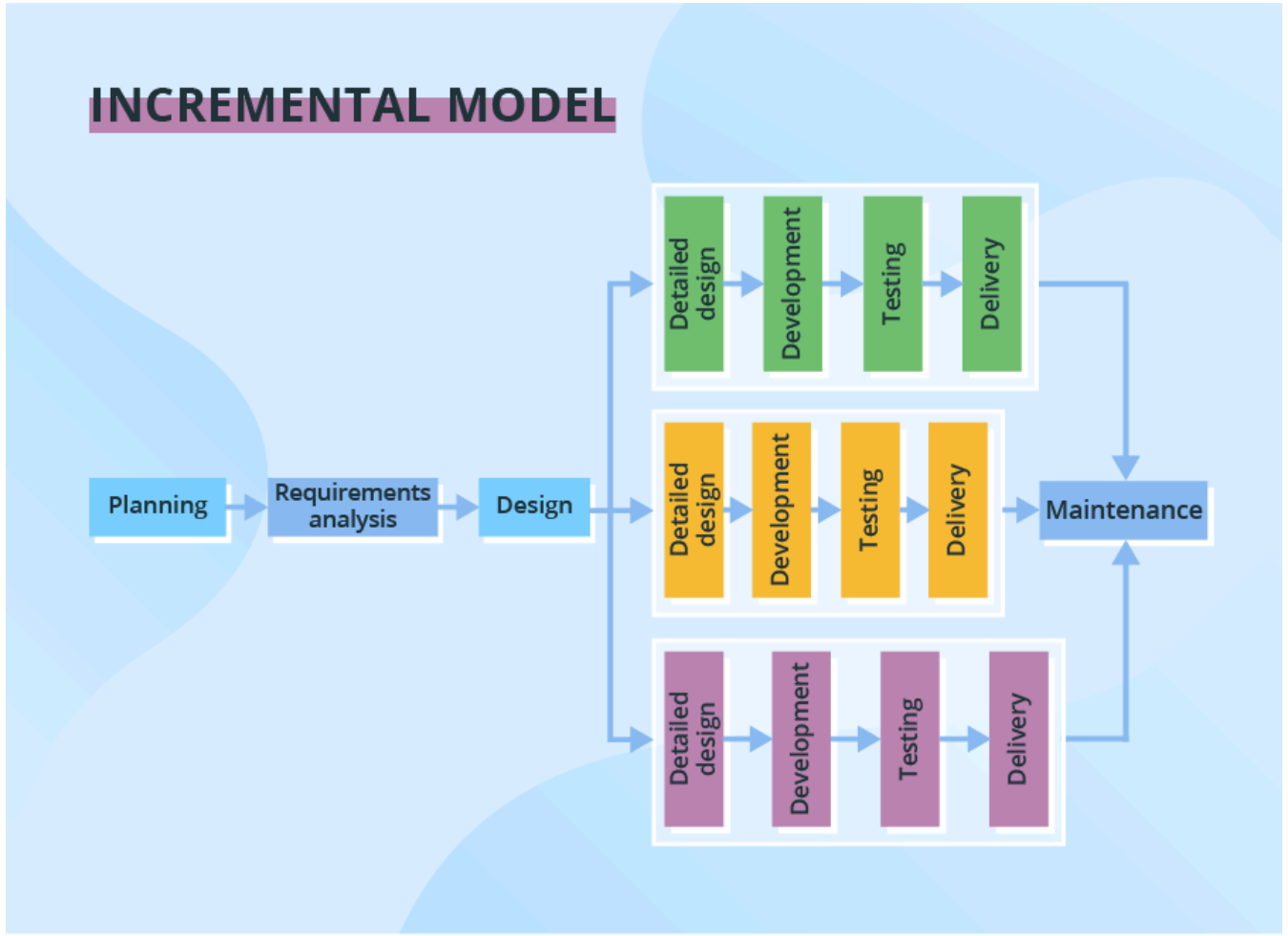
Best for

- High-risk enterprise solutions needing adaptive scope control plus governance.
- Programmes where component reuse and traceability are strategic.

Avoid when

- Teams are small, budgets lean, or a lightweight Agile flow would suffice.

© University of Southampton



Incremental Model – Flow

- Gather and prioritise baseline requirements.
- Slice the backlog into increments; each runs a mini waterfall (plan → design → build → test → deploy).
- Release increments sequentially, integrating and regression-testing the whole system each time.
- Capture user feedback after every drop to refine upcoming increments.

© University of Southampton

Incremental Model – Pros & Cons

Advantages

- Early usable functionality builds trust.
- Smaller batches reduce risk and simplify testing.
- Adapts to requirement shifts between increments.
- Lets stakeholders validate ROI before full investment.

Disadvantages

- Requires modular architecture to avoid rework.
- Multiple increments add coordination overhead.
- Duplicate efforts appear if teams lose the systems view.
- Scope creep emerges when change control is weak.

Best for

- Products with separable modules (authentication, payments, reporting).
- Organisations needing speed-to-market while keeping stability.

Avoid when

- Features are tightly coupled and only valuable when launched together.

© University of Southampton

RAD (Rapid Application Development) – Flow

1. **Business modelling:** Clarify processes, objectives, and success metrics.
2. **Data modelling:** Define entities, relationships, and governance rules.
3. **Process modelling:** Map workflows, decisions, and data flows.
4. **Application generation:** Use low-code tools, reusable components, and automation to prototype quickly.
5. **Testing & turnover:** Validate functionality continuously, then deploy with user buy-in.

© University of Southampton

RAD – Pros & Cons

Advantages

- Rapid prototyping shortens delivery cycles.
- Frequent user involvement boosts satisfaction.
- Automation + reuse can lower costs.

Disadvantages

- Demands skilled teams and supportive tooling.
- Less suitable for very large, complex programmes.
- Quality can slip if pace outruns governance.
- Dependent on mature low-code/automation platforms.

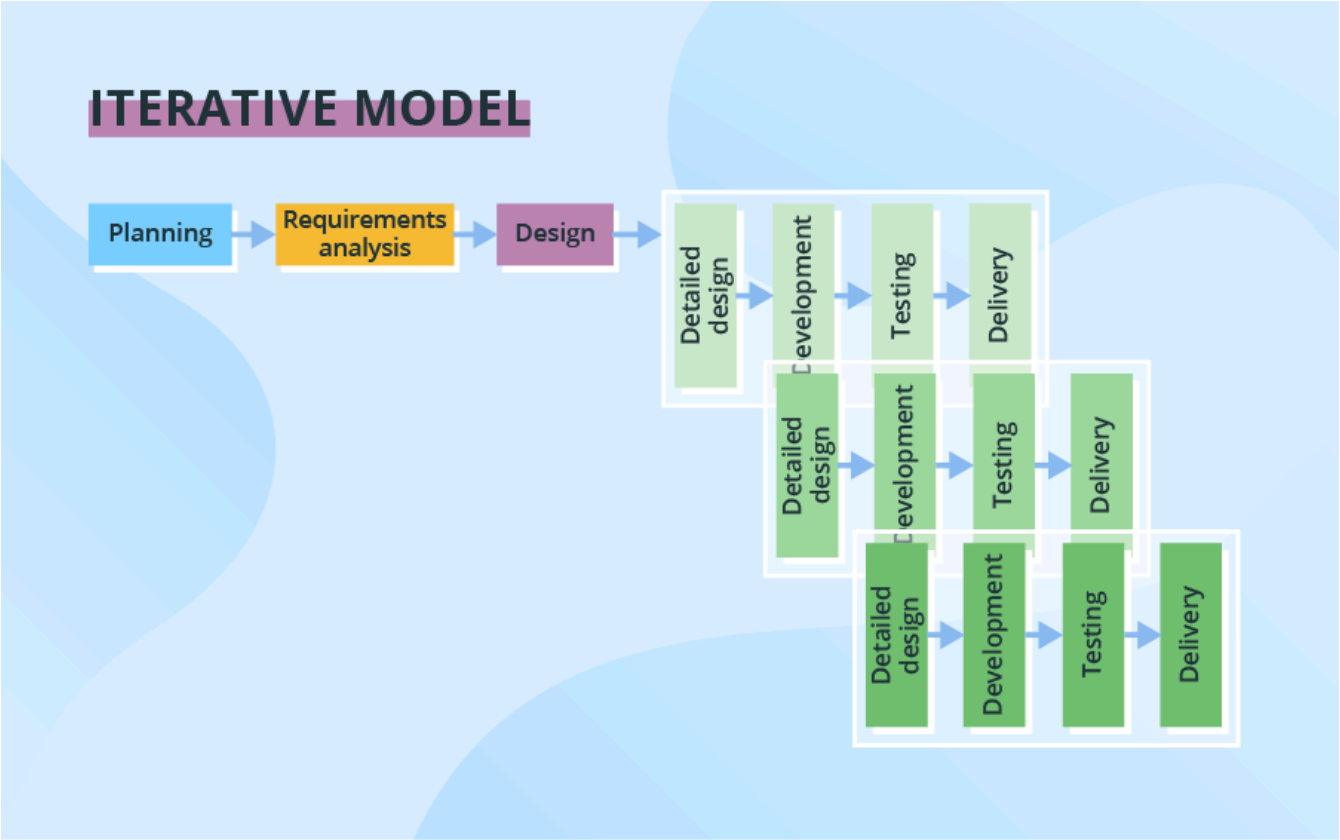
Best for

- Line-of-business apps where workflows can be modelled quickly.
- Programmes needing high user participation and fast iteration.

Avoid when

- Tooling is immature or the system is deeply complex/safety critical.

© University of Southampton



Iterative Model – Continuous Refinement

- Plan for a baseline architecture, then enhance the same solution in repeated loops.
- Each iteration revisits requirements, design, implementation, and testing, using feedback to steer the next slice.
- Design cohesion stays intact because new functionality builds on previous increments rather than shipping standalone modules.

© University of Southampton

Iterative Model – Pros & Cons

Benefits

- Flexibility—scope and priorities can evolve each cycle.
- Early issue detection thanks to repeated testing.
- Close stakeholder collaboration keeps value alignment.

Challenges

- Requires disciplined scope boundaries to avoid creep.
- Continuous client engagement is mandatory.
- Poor iteration planning elongates timelines.

Best for

- Products that must react to frequent stakeholder or market feedback.

Avoid when

- Comprehensive specifications are fixed and stakeholders prefer single delivery.

© University of Southampton

Prototype Model – Steps

1. **Identify key requirements** and UX expectations for the concept prototype.
2. **Design the prototype** with rapid tools, focusing on interactions and data.
3. **Build and demo** the prototype to stakeholders.
4. **Gather feedback** on usability, behaviour, and gaps.
5. **Iterate/refine** until stakeholders approve the direction.
6. **Harden into the final product**, adding full functionality, performance, and security.

© University of Southampton

Prototype Model – Pros & Cons

Advantages

- Early stakeholder involvement aligns expectations.
- Issues surface early, reducing downstream risk.
- Facilitates innovation when requirements are ambiguous.

Disadvantages

- Poorly scoped prototypes can balloon scope and cost.
- Users may confuse prototypes with production-ready software.
- Excess focus on UI can leave system qualities under-analysed.

Best for

- UX-heavy systems or ambiguous domains needing sense-making.
- Innovation/R&D tracks needing tangible artefacts to unlock funding.

Avoid when

- Users might mistake the prototype for the final deliverable or regulatory evidence is needed early.

© University of Southampton

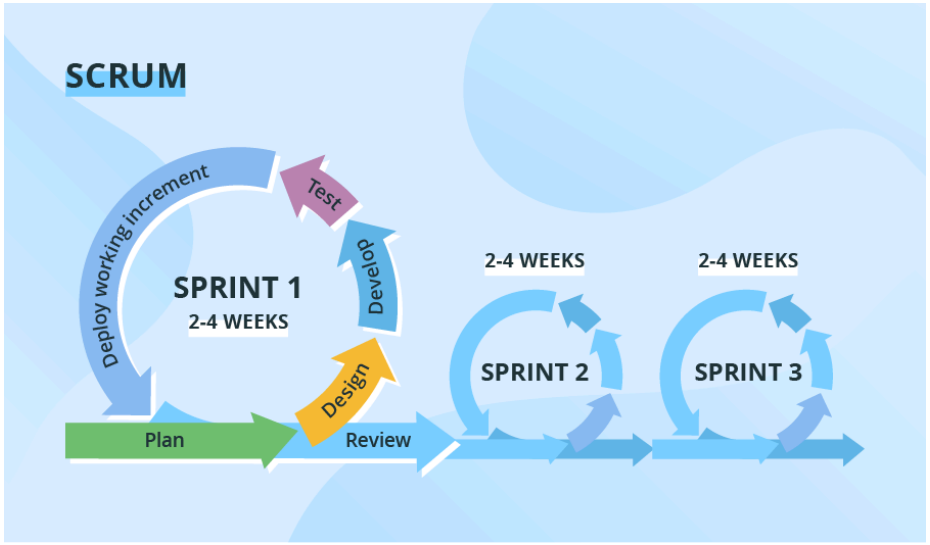
Agile Framework Spectrum

- **Scrum:** Time-boxed sprints, Scrum Master facilitation, and sprint reviews keep momentum while surfacing scope creep fast.
- **Kanban:** Continuous pull-based flow with WIP limits exposes bottlenecks and supports steady maintenance/ops work.
- **Extreme Programming (XP):** Engineering discipline (pairing, TDD, continuous integration) pushes build quality + feedback cadence.

Pick the framework that best matches stakeholder availability and the need for cadence vs. flow.

© University of Southampton

Scrum – Structured Sprints



- 2–4 week iterations with fixed goals; once a sprint starts, scope stays stable.
- Roles (Product Owner, Scrum Master, Dev Team) plus ceremonies (planning, daily stand-up, review, retro) keep alignment.
- Works best with 5–9 people to preserve communication density.

© University of Southampton

Benefits

- Transparent throughput and regular demos keep stakeholders engaged.
- Priorities reset each sprint, so ROI stays aligned to market feedback.

Best for

- Feature-driven products with stakeholders available for reviews.
- Teams of 5–9 people that can time-box work into 2–4 week sprints.

Challenges

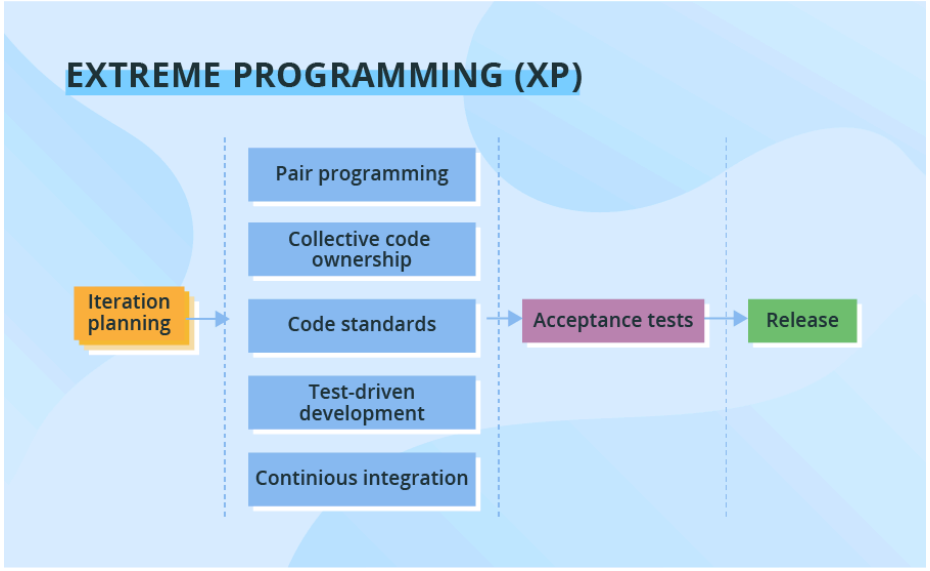
- Ceremony overhead grows if focus waivers or Agile mindset is missing.
- Scrum doesn't prescribe engineering practices—quality discipline must be layered on.

Avoid when

- Priorities shift daily or work doesn't produce frequent visible increments.
- Teams lack the discipline to uphold ceremonies and engineering quality.

© University of Southampton

Programming (XP) – Engineering Intensity



- 1–2 week iterations; user representatives (or Product Owners) must be on-call for immediate feedback.
- Practices include pair programming, test-driven development, continuous integration, collective code ownership, simple design, and small releases.

© University of Southampton

Benefits

- Maximises code quality, knowledge sharing, and collective ownership.
- Welcomes late-breaking changes as long as stories are unfinished.

Challenges

- Demands mature DevOps tooling plus close collaboration (often co-located teams).
- Resource intensive (pair programming, XP coach/tracker roles) and practised by only ~7% of Agile teams.

Best for

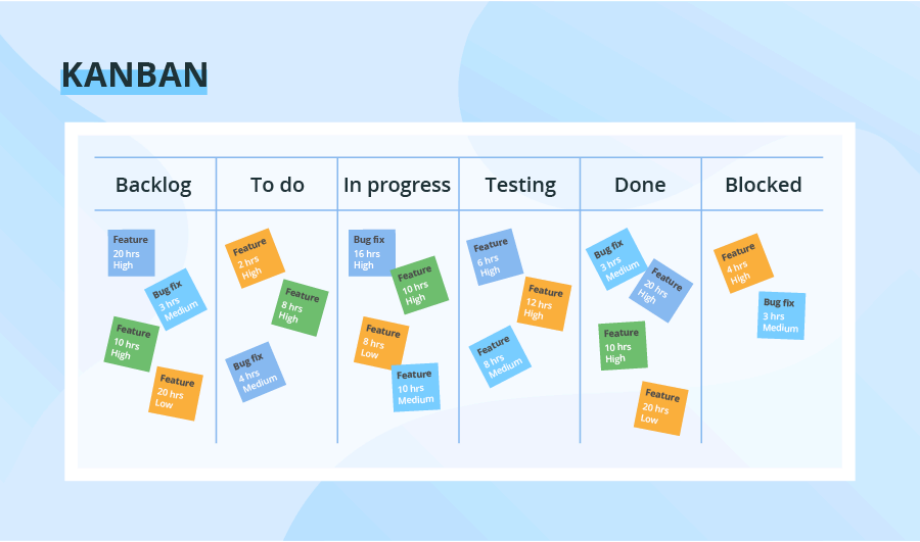
- Critical modules under active change, innovation spikes, or R&D work where quality can't slip.

Avoid when

- Teams are distributed, lack experienced engineers, or user/product proxies aren't available for rapid feedback.

© University of Southampton

Kanban – Continuous Flow



- Visual board with WIP limits; tasks pulled based on capacity rather than timeboxes.
- Work items can be reshuffled anytime, enabling quick response to incidents and support tickets.
- Meetings kept lightweight; emphasis on flow metrics (cycle time, throughput).

© University of Southampton

Benefits

- Minimal planning overhead; ideal for maintenance/operations.
- Transparency improves prioritisation and exposes bottlenecks.

Challenges

- Needs strong leadership and WIP discipline to prevent chaos.
- Without shared goals, teams may optimise local tasks yet miss outcomes.

Best for

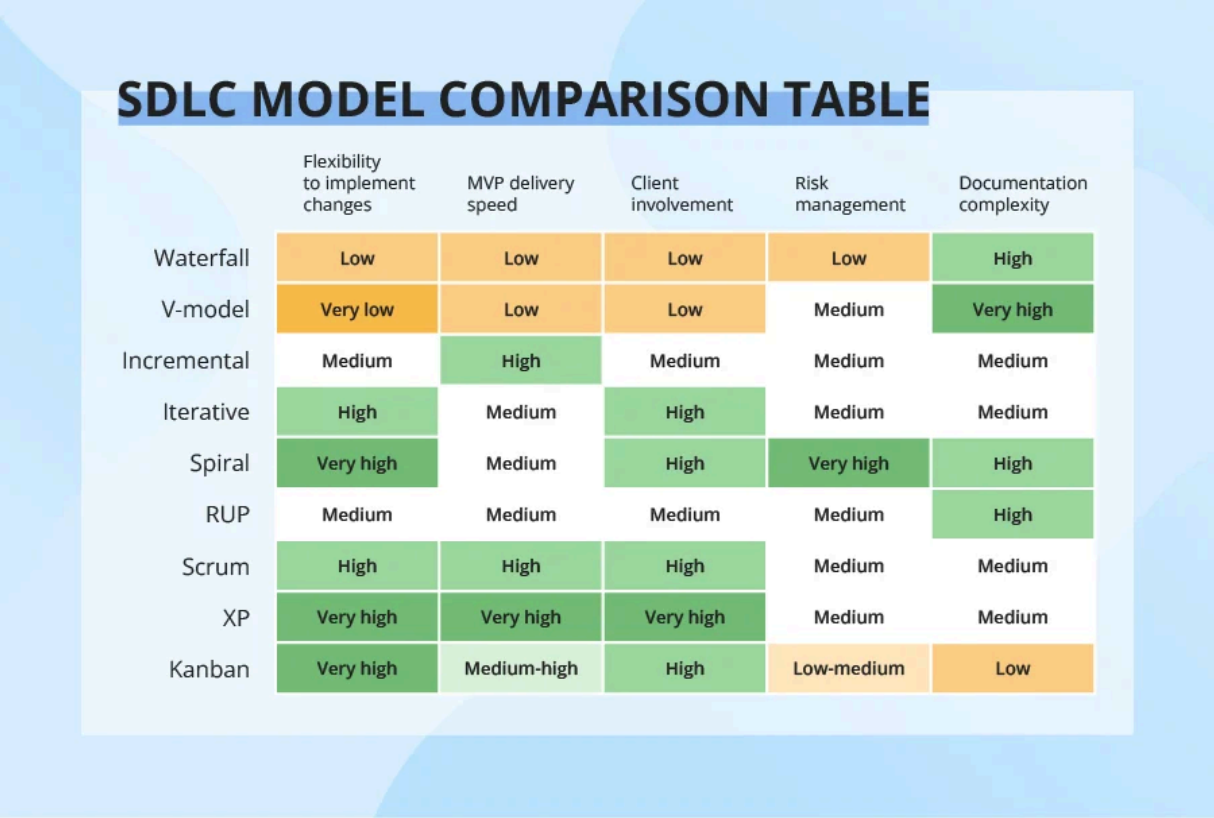
- Support, sustainment, DevOps, or teams handling unpredictable intake.

Avoid when

- Large new builds require coordinated releases across interdependent features.

© University of Southampton

Hybrid Approaches



SDLC MODEL COMPARISON TABLE

	Flexibility to implement changes	MVP delivery speed	Client involvement	Risk management	Documentation complexity
Waterfall	Low	Low	Low	Low	High
V-model	Very low	Low	Low	Medium	Very high
Incremental	Medium	High	Medium	Medium	Medium
Iterative	High	Medium	High	Medium	Medium
Spiral	Very high	Medium	High	Very high	High
RUP	Medium	Medium	Medium	Medium	High
Scrum	High	High	High	Medium	Medium
XP	Very high	Very high	Very high	Medium	Medium
Kanban	Very high	Medium-high	High	Low-medium	Low

Hybrid methodologies braid sequential and iterative strands: e.g., hardware follows Waterfall/V-Model gates, software services iterate via Agile/Incremental loops, and high-risk research inserts Spiral-style spikes. Synchronisation points (architecture reviews, integrated demos, compliance gates) keep the mixed cadences aligned.

© University of Southampton

Data Management Fundamentals

Data management governs how information is collected, stored, shared, and retired so requirements, decisions, and audits stay trustworthy. It spans people, process, and tooling choices that ensure data remains accurate, available, and secure throughout its lifecycle.

Think of it as the connective tissue between requirements, code, analytics, and stakeholders—without disciplined management, change requests are based on stale or incomplete evidence.

© University of Southampton

Data Management Practices

- **Governance:** Define ownership, access policies, metadata standards.
- **Quality:** Validate inputs, monitor drift, automate cleansing pipelines.
- **Lifecycle:** Plan ingestion → storage → archival or deletion with compliance in mind.
- **Tooling:** Use versioned data lakes, catalogs, and lineage trackers so teams know which dataset supports each requirement or test.
- **Security:** Apply least privilege, encryption, and monitoring aligned with regulatory obligations.

© University of Southampton

Git – Distributed Version Control

Git is a distributed version control system that records changes to files over time. Each developer holds a complete repository copy, enabling branching, merging, and offline history review—ideal for coordinating requirement-driven codebases across squads.

Key concept: commits create an immutable history with hashes linking changes to requirement IDs, tests, or stakeholder decisions.

© University of Southampton

Git Basics

Common workflow:

1. `git clone <repo>` to obtain the project history.
2. `git checkout -b feature/change-req` to isolate work.
3. Edit files, then `git add` and `git commit -m "REQ-42 update traceability"` to snapshot changes.
4. `git push` shares the branch; pull requests enable peer review.
5. `git merge` or `git rebase` integrates updates while preserving history.

Good hygiene: pull frequently, resolve conflicts transparently, and keep commits small so requirement changes map cleanly to code.

© University of Southampton

Advanced Source Control – Why It Matters

Modern software and hardware programmes rely on distributed teams working on tightly coupled systems. Monolithic development rarely scales; decomposition + coordination are mandatory. In source control, this means intentional branching, review rituals, and automation pipelines.

Systemic design principles remind us that source control is socio-technical: developers, testers, managers, and bots interact through shared tooling to keep change safe.

© University of Southampton

Branching Strategies

Branching decomposes work into parallel streams, mirroring distributed optimisation:

- **Feature branching:** Each feature owns a branch; reduces conflicts but increases integration effort later.
- **GitFlow:** Long-lived `develop` + `release` branches; excellent for regulated contexts yet slower for rapid iteration.
- **Trunk-based development:** Minimal branching with frequent commits to `main`; high velocity but demands strong CI/CD to avoid instability.

Case: Google enforces trunk-based development with automated bots guarding every commit, minimising integration cost via heavy automation investment.

© University of Southampton

Reviews as Socio-Technical Feedback

Code reviews operate as quality gates and learning loops:

- **Formal reviews:** Aerospace/medical domains tie reviews to compliance artefacts.
- **Peer reviews:** Open-source projects like the Linux kernel rely on expert maintainers.
- **Lightweight pull requests:** Agile teams optimise for speed while still enforcing standards.

Framed with systemic thinking, reviews are feedback loops—too rigid slows progress; too loose erodes quality. Balance depends on domain risk and stakeholder expectations.

© University of Southampton

Bots & Automation

Automation acts as an invisible workforce that scales source control:

- **Linting/style bots** (ESLint, Pylint) enforce readability.
- **Dependency bots** (Dependabot) surface updates and security fixes.
- **CI** orchestrates builds/tests per commit; **CD** pushes approved changes to environments.

Case: Microsoft Azure Repos relies on bots for ~70% of integration checks before human review, allowing people to focus on design trade-offs. Like meta-models in distributed design, bots reduce the cost of expensive evaluations.

© University of Southampton

Stakeholders & Systems Thinking

Source control workflows balance competing needs:

- **Developers:** low friction, rapid merges.
- **QA:** thorough testing + evidence.
- **Project managers:** traceability and predictable cadence.
- **Operations:** stability in deployment pipelines.

Systemic design identifies leverage points—e.g., pre-commit hooks or automated test suites—that produce large downstream impact by reducing review churn and deployment incidents.

© University of Southampton

Data + Source Control Feedback Loop



flowchart LR Reqs["Requirements & change requests"] --> DataMgmt["Data management\n(governance, quality, lineage)"] DataMgmt --> Git["Git workflows\n(branching + commits)"] Git --> Reviews["Reviews & bots\n(PRs, CI, automation)"] Reviews --> Deploy["Deployments & monitoring"] Deploy --> Insights["Stakeholder insights + telemetry"] Insights --> Reqs

Reinforces how disciplined data practices, Git hygiene, and automated reviews close the loop back to requirements, reducing cognitive load by showing the end-to-end system.

© University of Southampton

Summary

- Requirement change is inevitable—trace the ripples with systemic mapping, traceability, and stakeholder feedback loops.
- Predictive SDLCs (Waterfall, V-Model, RUP) deliver assurance when requirements are stable; risk-aware/iterative models (Spiral, Incremental, RAD, Prototype, Iterative) surface learning sooner.
- Agile frameworks (Scrum, XP, Kanban) plus hybrid orchestration keep multi-component programmes responsive without losing rigor.
- Governance levers—CCBs, matrices, compass-style selection—clarify when to switch tactics as uncertainty, risk, or regulation shifts.
- Tooling (data management, Git, automated reviews/bots) closes the loop, keeping changes auditable, observable, and safe-to-fail.

© University of Southampton